

PHASE 1 – Core Backend & Authentication

1. Introduction

Phase 1 is focused on building the **core backend and authentication system** for the Recruitment Mediator Platform. This phase is designed so that **even someone with no prior backend knowledge** can understand how the system is built, step by step.

The objective of this phase is to: - Set up Django properly - Connect it to a MySQL database - Create users with roles - Implement login and registration - Secure APIs using JWT - Control access using roles

No frontend or job logic is included in this phase.

2. Environment Setup

2.1 Python Installation

Python is required to run Django.

Command to verify Python installation:

```
python --version
```

If Python was not installed, it was downloaded from the official Python website and installed with **Add Python to PATH** enabled.

2.2 Django Installation

Django is the backend framework used.

Command used:

```
python -m pip install django
```

Verification:

```
python -m django --version
```

2.3 Django REST Framework Installation

Django REST Framework (DRF) is required to build APIs.

Command used:

```
python -m pip install djangorestframework
```

Then added in `project/settings.py`:

```
INSTALLED_APPS=[
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',
    'accounts',
]
```

3. Django Project Creation

3.1 Create Project

Command used:

```
python -m django startproject project
cd project
```

This created:

```
project/
├── project/
└── manage.py
```

3.2 Create App

Command used:

```
python manage.py startapp accounts
```

The `accounts` app handles authentication and users.

4. MySQL Database Setup

4.1 MySQL Installation

MySQL Server and MySQL Workbench were installed.

4.2 Create Database

Command used inside MySQL:

```
CREATE DATABASE project_db;
```

4.3 Install MySQL Driver

Command used:

```
python -m pip install mysqlclient
```

4.4 Django Database Configuration

In `project/settings.py`:

```
DATABASES={
    'default':{
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'project_db',
        'USER': 'root',
        'PASSWORD': 'your_password',
        'HOST': 'localhost',
        'PORT': '3306',
    }
}
```

5. Custom User Model

5.1 Why Custom User Model

Django default user was replaced to: - Use email instead of username - Add roles - Support future expansion

5.2 User Model Code

File: `accounts/models.py`

```
from django.contrib.auth.models import
AbstractBaseUser, PermissionsMixin, BaseUserManager
from django.db import models
from django.utils import timezone

class UserManager(BaseUserManager):
    def create_user(self, email, password=None, role='candidate'):
        if not email:
            raise ValueError('Email is required')
        user = self.model(
            email=self.normalize_email(email),
            role=role
        )
        user.set_password(password)
        user.save(using=self._db)
        return user

    def create_superuser(self, email, password=None):
        user = self.create_user(
            email=email,
            password=password,
            role='admin'
        )
        user.is_staff=True
        user.is_superuser=True
        user.save(using=self._db)
        return user

class User(AbstractBaseUser, PermissionsMixin):
    ROLE_CHOICES=(
        ('candidate', 'Candidate'),
        ('recruiter', 'Recruiter'),
        ('admin', 'Admin'),
    )

    email=models.EmailField(unique=True)
    role=models.CharField(max_length=20, choices=ROLE_CHOICES)
    is_active=models.BooleanField(default=True)
    is_staff=models.BooleanField(default=False)
    created_at=models.DateTimeField(default=timezone.now)

    objects=UserManager()

    USERNAME_FIELD='email'
```

```
def __str__(self):  
    return self.email
```

5.3 Tell Django to Use Custom User

In `project/settings.py`:

```
AUTH_USER_MODEL='accounts.User'
```

6. Migrations

Commands used:

```
python manage.py makemigrations  
python manage.py migrate
```

This created all required tables in MySQL.

7. JWT Authentication Setup

7.1 Install JWT Library

```
python -m pip install djangorestframework-simplejwt
```

7.2 JWT Configuration

In `project/settings.py`:

```
REST_FRAMEWORK={  
    'DEFAULT_AUTHENTICATION_CLASSES':(  
        'rest_framework_simplejwt.authentication.JWTAuthentication',  
    ),  
}
```

7.3 JWT URLs

In `project/urls.py`:

```
from django.contrib import admin  
from django.urls import path, include  
from rest_framework_simplejwt.views import
```

```
TokenObtainPairView,TokenRefreshView

urlpatterns=[
    path('admin/',admin.site.urls),
    path('api/',include('accounts.urls')),
    path('api/
login/',TokenObtainPairView.as_view(),name='token_obtain_pair'),
    path('api/token/
refresh/',TokenRefreshView.as_view(),name='token_refresh'),
]
```

8. Register API

Serializer

File: `accounts/serializers.py`

```
from rest_framework import serializers
from .models import User

class RegisterSerializer(serializers.ModelSerializer):
    password=serializers.CharField(write_only=True)

    class Meta:
        model=User
        fields=('email','password','role')

    def create(self,validated_data):
        user=User(
            email=validated_data['email'],
            role=validated_data['role']
        )
        user.set_password(validated_data['password'])
        user.save()
        return user
```

View

File: `accounts/views.py`

```
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from .serializers import RegisterSerializer

class RegisterView(APIView):
```

```

def post(self, request):
    serializer=RegisterSerializer(data=request.data)
    if serializer.is_valid():
        serializer.save()
        return Response({'message': 'User registered successfully'}, status=status.HTTP_201_CREATED)
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

```

9. Protected APIs and Role-Based Access

Permissions

File: `accounts/permissions.py`

```

from rest_framework.permissions import BasePermission

class IsRecruiter(BasePermission):
    def has_permission(self, request, view):
        return request.user.role=='recruiter'

class IsCandidate(BasePermission):
    def has_permission(self, request, view):
        return request.user.role=='candidate'

class IsAdmin(BasePermission):
    def has_permission(self, request, view):
        return request.user.role=='admin'

```

Views

```

from rest_framework.permissions import IsAuthenticated
from .permissions import IsRecruiter, IsCandidate, IsAdmin

class ProtectedView(APIView):
    permission_classes=[IsAuthenticated]
    def get(self, request):
        return Response({'message': 'JWT is working'})

class RecruiterOnlyView(APIView):
    permission_classes=[IsAuthenticated, IsRecruiter]
    def get(self, request):
        return Response({'message': 'Recruiter access granted'})

class CandidateOnlyView(APIView):
    permission_classes=[IsAuthenticated, IsCandidate]
    def get(self, request):
        return Response({'message': 'Candidate access granted'})

```

```
class AdminOnlyView(APIView):
    permission_classes=[IsAuthenticated,IsAdmin]
    def get(self,request):
        return Response({'message':'Admin access granted'})
```

10. URLs

File: `accounts/urls.py`

```
from django.urls import path
from .views import
RegisterView,ProtectedView,RecruiterOnlyView,CandidateOnlyView,AdminOnlyView

urlpatterns=[
    path('register/',RegisterView.as_view(),name='register'),
    path('protected/',ProtectedView.as_view(),name='protected'),
    path('recruiter/',RecruiterOnlyView.as_view(),name='recruiter'),
    path('candidate/',CandidateOnlyView.as_view(),name='candidate'),
    path('admin/',AdminOnlyView.as_view(),name='admin'),
]
```

11. Admin Panel Setup

Register Model

File: `accounts/admin.py`

```
from django.contrib import admin
from .models import User

admin.site.register(User)
```

Create Superuser

Command:

```
python manage.py createsuperuser
```

Access Admin Panel

URL:

`http://127.0.0.1:8000/admin/`

12. Final Outcome

Phase 1 successfully delivers: - Secure Django backend - MySQL database integration - Custom user model with roles - Registration and login APIs - JWT authentication - Role-based access control - Admin management panel

Phase 1 establishes a complete and secure system foundation.